

Design Document: Jackson Investments Financial Advisor Bot

Table of Contents

Design Document: Jackson Investments Financial Advisor Bot 1

1. Domain & Users 1

2. Software Architecture 1

 Key Design Decisions..... 3

3. Detailed System Architecture 4

 3.1 LSTM Signal Generator 4

 3.2 RL Agent 5

 3.3 Frontend-Backend Integration 7

4. UI Design..... 8

5. Evaluation Plan 10

 5.1 Model Evaluation 10

 5.2 User Study 11

References 11

1. Domain & Users

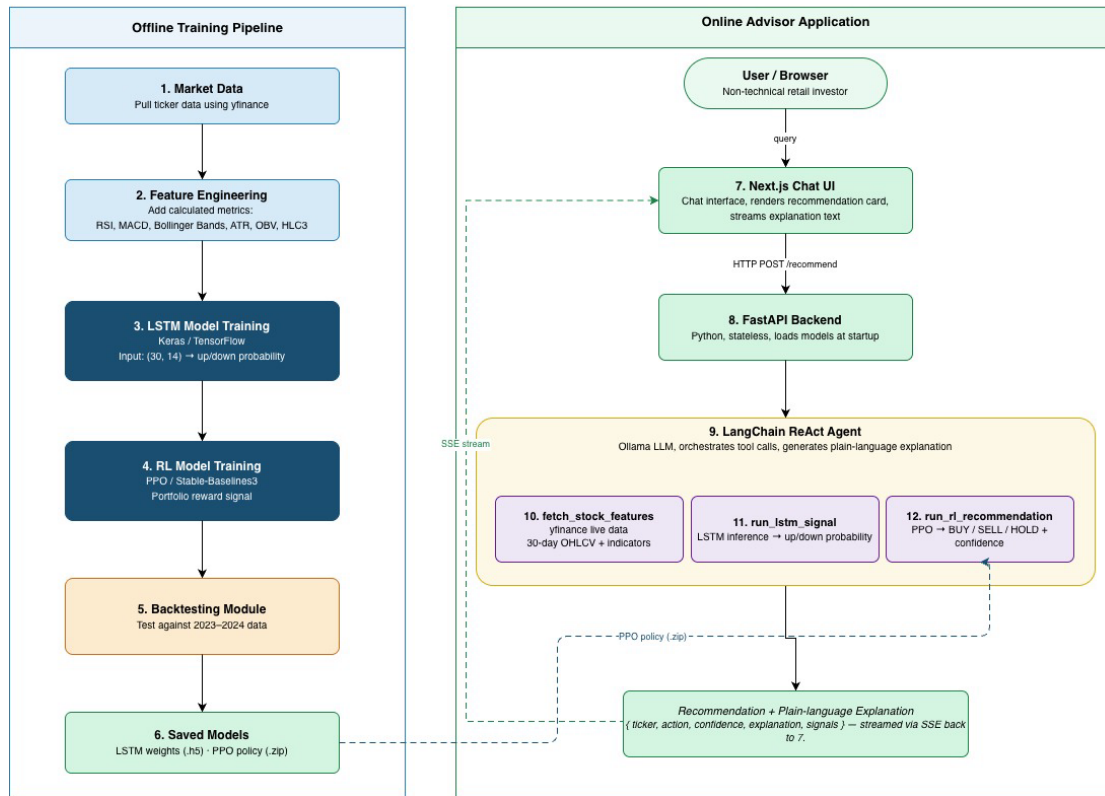
The target user is a non-technical retail investor managing a portfolio of equities listed on the TSX, NYSE, and NASDAQ exchanges. They understand what stocks they own but do not have the time, expertise, or tools to analyse market signals across a portfolio as a whole.

Existing tools, such as Danelfin, use decision trees applied to individual stocks without timeframe context, and do not account for portfolio-level state. Fully integrated LSTM-RL systems (Jiang et al., 2017; Zou et al., 2024) have shown strong performance but are not accessible to non-technical users via a natural language interface. This project bridges that gap by combining a two-stage AI architecture with a plain-language advisory layer.

2. Software Architecture

The system is split into an offline training pipeline and an online advisor application.

Jackson Investments — System Architecture



Offline Training Pipeline (left)

#	Component	Description
1	Market Data	yfinance downloads historical OHLCV data for selected equities
2	Feature Engineering	I transform raw price data into the 14 model features: OHLCV, HLC3, RSI, MACD, MACD Signal, Bollinger Bands (upper/mid/lower), ATR, and OBV
3	LSTM Model Training	Stage 1 LSTM model trained in Keras/TensorFlow. Input shape (30, 14); output is a directional probability (up/down) per stock per day
4	RL Model Training	Stage 2 PPO model trained in Stable-Baselines. Uses LSTM signals and portfolio state as inputs; reward is daily change in portfolio value
5	Backtesting Module	I evaluate trained models on 2023-2024 data against buy-and-hold and random-policy baselines.
6	Saved Models	The pipeline saves trained LSTM weights (.h5) and PPO policy (.zip) to disk. The online application loads them at startup

Online Advisor Application (right)

#	Component	Description
7	Next.js Chat UI	Browser-based chat interface. The user types a natural-language question about a stock or portfolio. Recommendation cards and streamed explanation text are rendered here
8	FastAPI Backend	Python backend that receives requests from 7, initialises the LangChain agent, and streams the response back via Server-Sent Events (SSE)
9	LangChain ReAct Agent	Ollama LLM acting as the orchestration layer. It calls tools 10-12 in sequence, then generates the plain-language explanation from the structured tool outputs. It does not make investment decisions independently
10	fetch_stock_features	LangChain tool that pulls 30 days of live market data from yfinance and computes the 14 input features
11	run_lstm_signal	LangChain tool that runs LSTM inference using weights loaded from 6 and returns a up/down probability
12	run_rl_recommendation	LangChain tool that runs PPO inference using the policy loaded from 6 and returns a BUY/SELL/HOLD action and confidence score

Dashed arrows between 6 and 11/12 indicate that the saved model weights are loaded into the inference tools at FastAPI startup. The final response is streamed from 9 back to 7 via SSE.

Key Design Decisions

Separate LSTM and RL rather than integrated (CLSTM-PPO style) Zou et al. (2024) train both layers end-to-end as a single system. I want to train them separately so I can debug the LSTM output separate from the RL output. The trade-off is that the LSTM may not be optimised to produce output that helps the RL agent, so combined performance may be slightly lower than a fully integrated approach. The benefit is a more interpretable system where I can evaluate each stage on its own terms.

PPO for the RL agent I chose Proximal Policy Optimisation based on Zou et al. (2024), who found it outperformed A2C and DDPG on equity markets including the Dow Jones 30 and FTSE 100. It also has stable, well-documented implementations in Stable-Baselines3.

30-day lookback window for Stage 1 Zou et al. (2024) found that a 30-day rolling window for the time-based encoder produced the best results in their CLSTM-PPO experiments. Wei (2019) also found that shorter windows introduce a time-lag effect where predicted peaks and troughs trail real ones which would produce stale signals. I will use a 30-day window as the starting point.

Ollama for explanation only I configure Ollama to handle explanation only. It receives the structured output from the RL agent (recommendation, confidence, contributing signals) and rewrites it as a plain-language explanation. This keeps the decision logic fully traceable while meeting the requirement for non-technical users.

3. Detailed System Architecture

3.1 LSTM Signal Generator

The LSTM model takes a rolling window of 30 trading days for a single stock as input and outputs the probability that the closing price will be higher on the following day than it is today. A probability above 0.5 is treated as a bullish signal and below 0.5 it is bearish. Zou et al. (2024) identified that a 30-day window is the optimal lookback period in their CLSTM-PPO experiments, and Wei (2019), showed that shorter windows introduce a lag effect that makes signals stale.

Input features

Each of the 30 days in the input window is represented by a vector of features calculated from the base OHLCV data. Mostafavi and Hooman (2025) used SHAP analysis across 73 years of S&P 500 data to identify the most predictive features. From their list I selected these features:

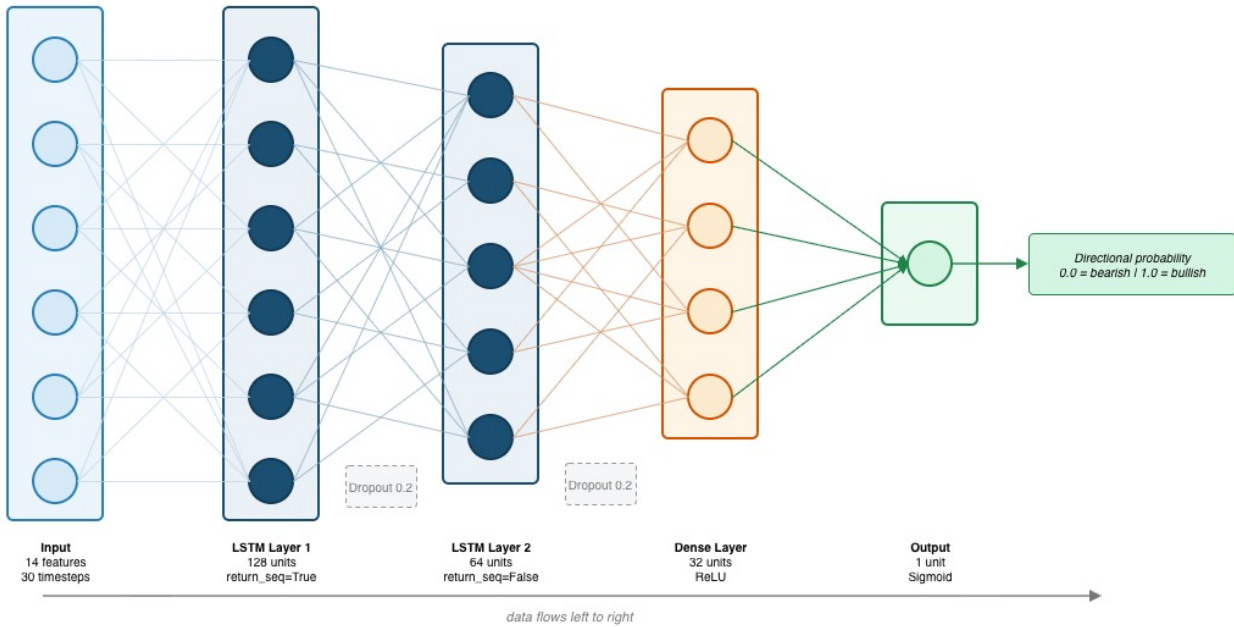
Feature	Type	Notes
Open, High, Low, Close, Volume	Price / volume	Raw OHLCV values, normalised
HLC3	Momentum	Average of High, Low, Close
RSI (14-day)	Momentum	Relative Strength Index
MACD and MACD Signal	Trend	12/26/9 standard settings
Bollinger Bands (upper, mid, lower)	Volatility	20-day, 2 std dev
ATR (14-day)	Volatility	Average True Range
OBV	Volume	On-Balance Volume

This gives an input shape of (30 timesteps × 14 features) per stock per day.

Network architecture

I will build the LSTM model in Keras with the following layer structure:

LSTM Signal Generator - Network Architecture



I will use 2 LSTM layers rather than 1 to let the model learn both short-term and longer-term patterns within the 30-day window. I will apply a dropout of 0.2 after each LSTM layer to reduce overfitting. The output is a single sigmoid value from 0 to 1 representing bearish to bullish.

I will train a single LSTM model using binary cross-entropy loss with the Adam optimiser at a learning rate of 0.0001 and a batch size of 64. The model is trained on concatenated windowed data from multiple stock tickers. This will allow it to learn patterns across different volatilities and trading characteristics.

3.2 RL Agent

I will implement the RL agent using Proximal Policy Optimisation (PPO) from the Stable-Baselines3 library. It receives the current LSTM signals and portfolio state and outputs an action for each stock in the portfolio: Buy, Sell, or Hold.

Environment

I will use a custom OpenAI Gym environment to simulate the portfolio. At each daily timestep, the environment gives the agent an observation, receives an action, applies it to the simulated portfolio, and returns a reward.

Observation space

For a portfolio of N stocks, the observation at each timestep is a vector containing:

Component	Size	Description
LSTM signal per stock	N	Directional probability from Stage 1 (0-1)

Component	Size	Description
Current position per stock	N	Number of shares currently held, normalised
Current price per stock	N	Latest closing price, normalised
Available cash ratio	1	Cash available compared to total portfolio value
Total portfolio value	1	Current total value, normalised

Total observation size: **(3N + 2)**. For an initial portfolio of 5 stocks this gives a 17-dimensional state vector.

Action space

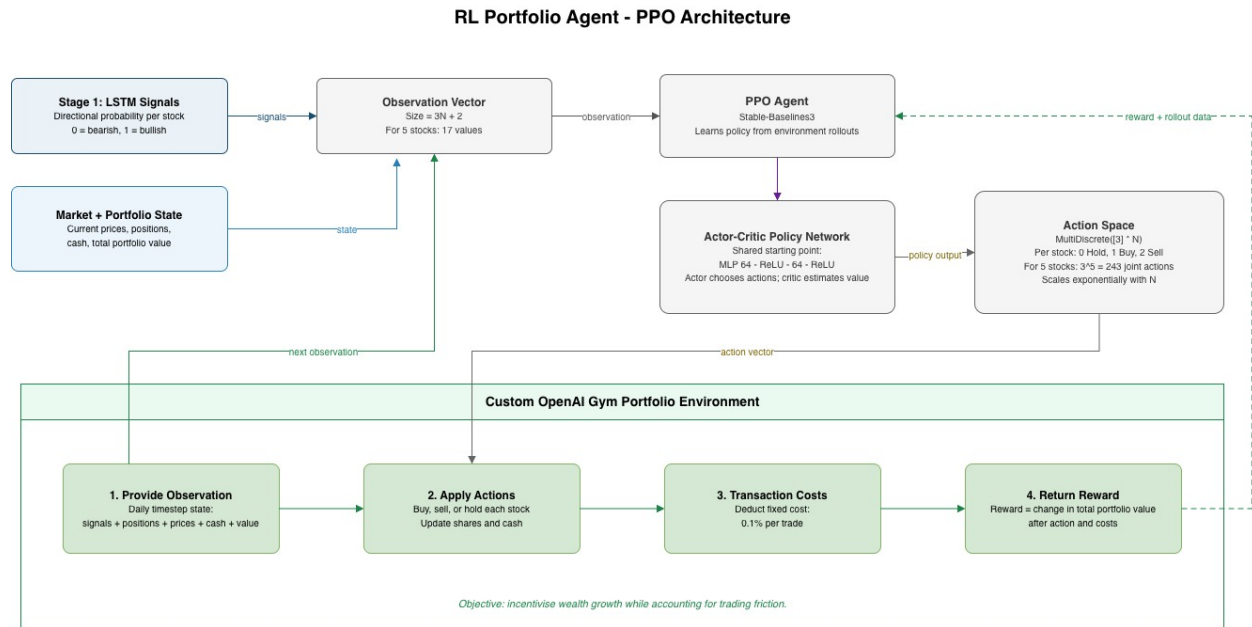
PPO works well with a MultiDiscrete action space. It will use one of 3 actions (0 = Hold, 1 = Buy, 2 = Sell) per stock per timestep. For 5 stocks this produces $3^5 = 243$ possible action combinations which is fine however the number of actions grows exponentially so it would become a problem to have a large portfolio.

Reward function

The reward at each timestep is the change in total portfolio value after applying the action and deducting a fixed transaction cost of 0.1% per trade. This is consistent with the cost structure used by Jiang et al. (2017). Using portfolio value change as the reward means the agent is incentivised to grow wealth.

Policy network

I will use a two-layer MLP with 64 units per layer and ReLU activations for both the actor and critic networks. This is the default Stable-Baselines3 architecture and I will use it as the starting point before any hyperparameter tuning.



3.3 Frontend-Backend Integration

The Next.js frontend and the Python backend communicate over a REST API. The backend is a FastAPI application that loads the trained LSTM and PPO models at startup. I will use a LangChain ReAct agent as the orchestration layer. The agent receives the user's natural-language query, decides which tools to call and in what order, and produces the plain-language explanation as its final response, using the structured tool outputs as its context. The LSTM and RL models will both be wrapped as tools.

This design keeps the investment decision logic entirely within the deterministic tools. The LLM never makes a buy, sell, or hold judgement. It only synthesises and explains results that the tools have already produced.

LangChain agent tools

I will register 3 tools with the agent:

Tool	Input	Output
fetch_stock_features(ticker)	Ticker symbol	30-day OHLCV + indicator matrix, cached by date
run_lstm_signal(ticker, features)	Ticker + feature matrix	Directional probability (float 0-1)
run_rl_recommendation(ticker, signal, portfolio_state)	Ticker + signal + portfolio context	Action (BUY/SELL/HOLD), confidence score

I will initialise the agent with a system prompt that instructs it to always call all 3 tools before producing a response and restrict it from reasoning about the investment decision independently. This ensures the tool-calling order while retaining the flexibility of the natural-language interface.

API endpoints

Method	Endpoint	Description
POST	/recommend	Takes a query string and portfolio state; streams a single stock recommendation via SSE
POST	/portfolio	Takes the full portfolio; runs the agent once per stock and returns an aggregated summary
GET	/health	Returns backend status and whether models are loaded

Portfolio state management

I store the user's portfolio (list of tickers and share quantities) in the Next.js frontend as local state and pass it with each request. No portfolio data is persisted server-side in the initial version. This keeps the backend stateless and simplifies deployment.

4. UI Design

The background research and the assignment brief both describe that the system must be usable by non-technical investors. This creates a specific design challenge: the underlying models produce structured, numerical outputs (signal probabilities, portfolio weights, Sharpe ratios) that are meaningless to a non-technical user without translation. At the same time, oversimplifying the output risks producing advice the user cannot critically evaluate or trust.

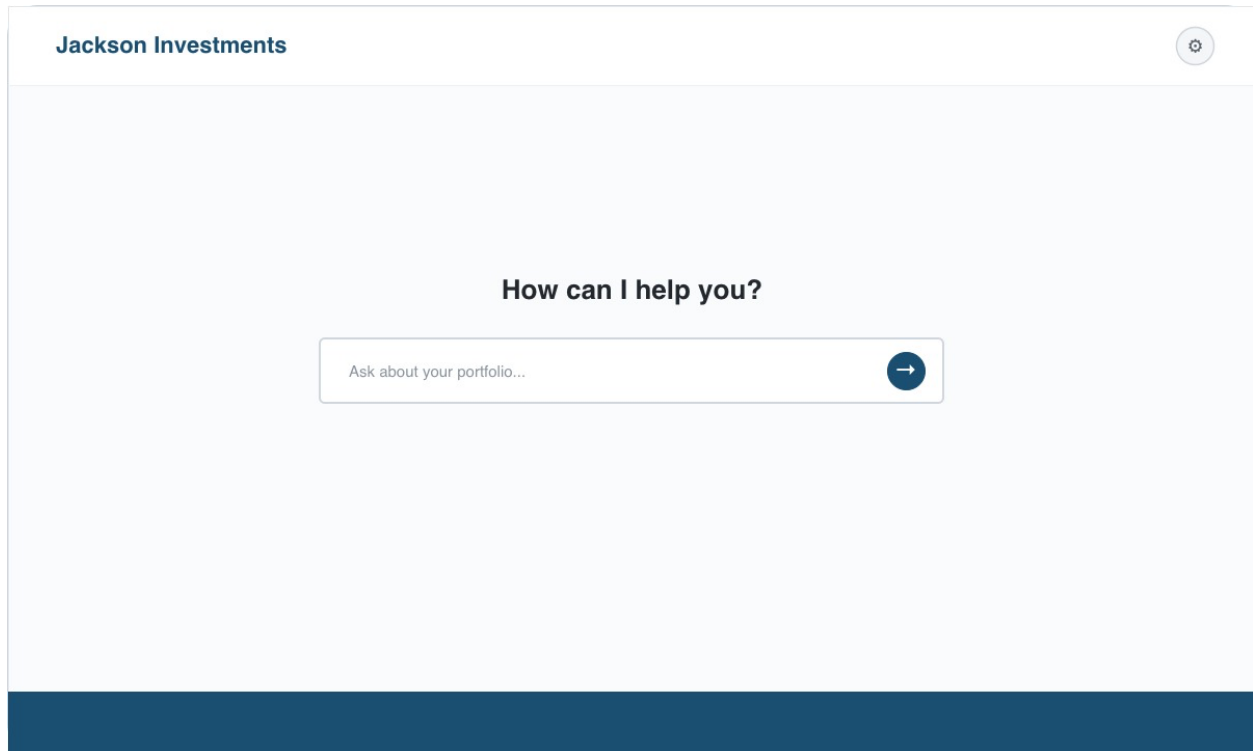
The design will be a layered interface with 3 levels of detail. The first level gives a clear recommendation and a short plain-language reason. The second level shows a confidence score. The third level exposes the raw model signals (LSTM probability, key indicators) for users who choose to dig deeper. This progressive disclosure pattern ensures the interface is approachable for non-technical users without hiding information from those who want it.

I chose a chat-style interface over a dashboard layout for 2 reasons. First, asking a question about a specific stock is a more natural interaction for a non-technical user than navigating a multi-panel dashboard with charts. Second, a chat interface lets the user ask follow-up questions and receive contextualised responses, rather than interpreting data independently.

The interface has 2 main screens.

Screen 1: Query Input

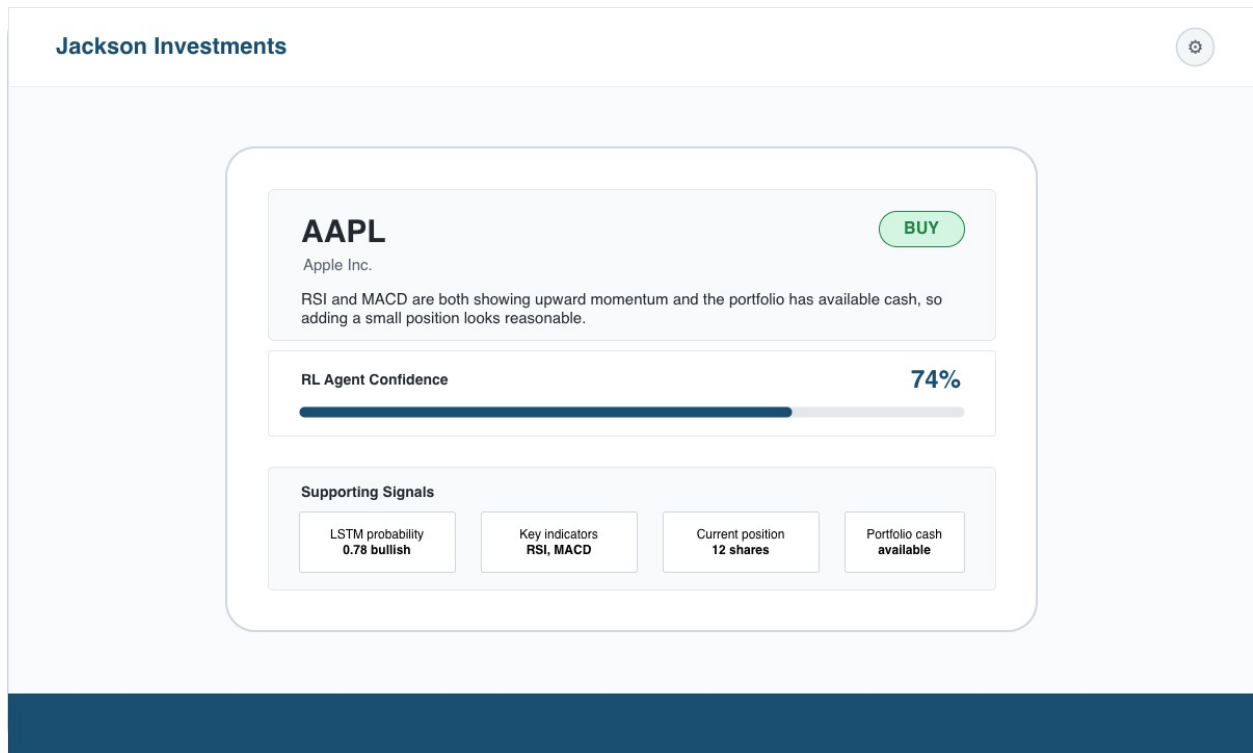
The user types a natural-language question about a stock or their portfolio (e.g. "Should I buy more AAPL?" or "How does my portfolio look today?"). The input field accepts both ticker symbols and company names.



Screen 2: Recommendation Output

The response is displayed in 3 layers:

- **Layer 1:** Stock ticker, company name, recommendation badge (BUY / SELL / HOLD), and a one-sentence plain-language explanation generated by the Ollama layer (e.g. "RSI and MACD are both showing upward momentum and the portfolio has available cash, so adding a small position looks reasonable.")
- **Layer 2:** Confidence indicator showing the RL agent's output probability as a percentage (e.g. 74% confidence).
- **Layer 3:** Supporting signals panel showing the raw LSTM directional probability, the key technical indicators that contributed to the signal, and the current portfolio position for that stock.



5. Evaluation Plan

The aim of the evaluation is to assess whether the system successfully gives non-technical investors clear, accurate, and portfolio-aware buy, sell, or hold recommendations. The evaluation has 2 parts: a quantitative backtesting evaluation of the AI models, and a user study evaluating whether the advisory output is understandable and usable.

5.1 Model Evaluation

I will evaluate the LSTM and RL agent separately before assessing them as a combined system. This lets me isolate where performance problems originate.

I will use data from January 2015 to December 2022 for all model training. The held-out test period will be January 2023 to December 2024. I will not use any data from the test period during training or hyperparameter tuning.

Stage 1: LSTM Signal Generator

I will evaluate the LSTM on its ability to predict the direction of next-day price movement (up or down) on the held-out test set. Following the methodology used by Mostafavi and Hooman (2025), I will measure performance using:

- Directional accuracy: the percentage of test days where the predicted direction matches the actual direction
- Mean Absolute Error (MAE) on next-day closing price
- Comparison against a naïve baseline that predicts the previous day's close as tomorrow's close

I use 55% directional accuracy as the threshold for the signal to be considered useful, based on the benchmarks reported in Zou et al. (2024).

Stage 2: RL Agent Portfolio Performance

I will evaluate the PPO agent by running it on the held-out test period using the LSTM signals generated from the same period. I will compare it against 2 baselines:

- A buy-and-hold strategy (invest equally across all stocks at the start of the test period and hold)
- A random policy (buy, sell, or hold chosen uniformly at random)

I chose buy-and-hold as the minimum bar because it represents the simplest strategy a retail investor could use without any AI assistance. I will record the following metrics for all 3:

- Cumulative portfolio return over the test period
- Sharpe ratio (annualised return relative to volatility)
- Maximum drawdown (the largest peak-to-trough decline)

5.2 User Study

The user study will assess whether non-technical users can understand and act on the recommendations the system produces, and whether the interface is usable. I will recruit 5 participants who self-identify as non-technical investors (i.e. they own or are interested in stocks but have no background in finance or machine learning).

I will give each participant a scenario: they hold a portfolio of 3 stocks and have a small amount of cash available. They will then use the live web interface to ask the advisor bot about each stock in turn and ask for an overall portfolio view.

After each recommendation is displayed, I will ask each participant 3 comprehension questions:

1. What does the system recommend you do with this stock?
2. What is the main reason given for that recommendation?
3. How confident is the system in this recommendation?

These questions check whether participants can extract the key information from the plain-language output. I will mark a response correct if it matches the structured model output that generated the explanation.

References

- Jiang, Z., Xu, D., & Liang, J. (2017). A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem. arXiv:1706.10059.
- Ding, G., & Qin, L. (2020). Study on the prediction of stock price based on the associated network model of LSTM. *International Journal of Machine Learning and Cybernetics*, 11(6), 1307–1317.
- Zou, J., Lou, J., Wang, B., & Liu, S. (2024). A novel Deep Reinforcement Learning based automated stock trading system using cascaded LSTM networks. *Expert Systems with Applications*, 242, 122801.

- Mostafavi, S. M., & Hooman, A. R. (2025). Key technical indicators for stock market prediction. *Machine Learning with Applications*, 20, 100631.
- Wei, D. (2019). Prediction of Stock Price Based on LSTM Neural Network. *2019 International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM)*, 544–547.